

PROGRAM

```
#include <stdio.h>

int heap[20], n = 0;

void insert(int value)
{
    int i, parent, temp;

    heap[n] = value;
    i = n;
    n++;

    while (i > 0)
    {
        parent = (i - 1) /
2;

        if (heap[parent] >=
heap[i];
            heap[i] = temp;

            i = parent;
        }
    }

void display()
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ",
heap[i]);
    printf("\n");
}

int main()
{
    int a[] = {45, 72, 30,
90, 65, 50, 85};
    int i;

    printf("Max Heap after
each insertion:\n");

    for (i = 0; i < 7; i++)
    {
        insert(a[i]);
        printf("After
inserting %d: ", a[i]);
        display();
    }

    return 0;
}
```

OUTPUT

Max Heap after each
insertion:

After inserting 45: 45

After inserting 72: 72 45

After inserting 30: 72 45
30

After inserting 90: 90 72
30 45

After inserting 65: 90 72
30 45 65

After inserting 50: 90 72
50 45 65 30

After inserting 85: 90 85
50 45 65 30 72

PROGRAM

```
#include <stdio.h>

void heapify(int a[], int n, int i)
{
    int largest = i, l = 2*i+1, r = 2*i+2, temp;

    if (l < n && a[l] > a[largest])
        largest = l;

    if (r < n && a[r] > a[largest])
        largest = r;

    if (largest != i)
    {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n)
{
    int i, temp;

    for (i = n/2-1; i >= 0; i--)
        heapify(a, n, i);

    for (i = n-1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main()
{
    int a[] = {45, 72, 30, 90, 65, 50, 85};
    int n = 7, i;

    heapSort(a, n);

    printf("Sorted array: ");

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OUTPUT

Sorted array: 30 45 50 65
72 85 90

PROGRAM

```
#include <stdio.h>

int comparisons = 0, swaps = 0;

void display(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;
    int j, temp;

    for (j = low; j < high; j++)
    {
        comparisons++;

        if (a[j] < pivot)
        {
            i++;

            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            swaps++;
        }
    }

    temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;
    swaps++;

    return i + 1;
}

void quickSort(int a[], int low, int high, int n)
{
    if (low < high)
    {
        int p = partition(a, low, high);

        printf("Pivot %d:", a[p]);
        display(a, n);

        quickSort(a, low, p - 1, n);
        quickSort(a, p + 1, high, n);
    }
}
```

```
}

int main()
{
    int a[] = {45, 72, 30, 90, 65, 50, 85};
    int n = 7, i;

    printf("Quick Sort Trace:\n");

    quickSort(a, 0, n - 1, n);

    printf("\nSorted array:");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nComparisons = %d", comparisons);
    printf("\nSwaps = %d\n", swaps);

    return 0;
}
```

OUTPUT

```
Quick Sort Trace:
Pivot 85: 45 72 30 65 50 85
90
Pivot 50: 45 30 50 65 72 85
90
Pivot 30: 30 45 50 65 72 85
90
Pivot 72: 30 45 50 65 72 85
90

Sorted array: 30 45 50 65
72 85 90
Comparisons = 12
Swaps = 12
```